

AI6126: Project 2 - Blind Face Super-Resolution Report

Cui Boning [G2403305J]

✉ CUI002@e.ntu.edu.sg

April 25, 2025

1 Data Processing

The methods used to read images into tensors, create data loaders, and generate and apply degradation matrices adhere to the same configurations given in *ffhqsub_dataset.py* and *train_SRResNet_x4_FFHQ_300k.yml*.

2 Model

2.1 Overview

The model used in this project is a Residual Channel-Attention Network (RCAN). A shallow head convolution extracts initial features from the $4 \times$ down-sampled input, a body of residual channel-attention blocks (each combining two 3×3 convolutions with a squeeze-and-excite module) iteratively refines them while local and global skip connections stabilize training, and a tail with two pixel-shuffle stages upsamples the enriched feature maps back to the high-resolution image. The network is randomly initialized and trained end-to-end from scratch under a reconstruction loss.

This model has a total number of **1,824,595** parameters.

2.2 Components

- **Head (Shallow-Feature Extractor):** A single 3×3 convolution maps the 3-channel low-resolution image to 64 feature channels (stride 1, padding 1). This lightweight front end captures basic edges and color cues and hands a 64-channel tensor to the deeper network without changing spatial resolution.
- **Residual Channel-Attention Block (RCAB):** Each RCAB contains two 3×3 convolutions ($64 \rightarrow 64$ channels) separated by a ReLU, followed by a channel-attention gate (CALayer). The block output is added back to its input, forming a local residual path that eases optimization while letting the convolutions focus on high-frequency detail.

- **CALayer (Channel-Attention Gate):** Inside every RCAB, global average pooling squeezes the feature map to a 64-element descriptor. A two-layer 1×1 MLP ($64 \rightarrow 4 \rightarrow 64$ channels, reduction ratio 16) produces a sigmoid gating vector that re-weights each feature channel. This “squeeze-and-excite” operation lets the network amplify informative channels and suppress less useful ones adaptively for each image.
- **Body (Deep Feature Refinement + Global Skip):** Twenty RCABs are stacked sequentially, followed by an additional 3×3 convolution that mixes their outputs. A global skip connection then adds the head’s shallow features to this deep feature tensor, preserving low-frequency information and encouraging residual learning across the entire depth of the network.
- **Tail (Progressive Upsampling and Reconstruction):** Two upsampling stages turn the 64-channel features into the final high-resolution image. Each stage applies a 3×3 convolution that expands channels from 64 to 256, then a PixelShuffle layer that rearranges them to double height and width ($\times 2$). After the second stage the spatial size is $4 \times$ the input, and a final 3×3 convolution projects the 64 channels back to 3 RGB channels, producing the super-resolved output.

The full implementation is given in Appendix A.

3 Training Pipeline

3.1 Hyperparameters

- **epoch:** 200;
- **batch_size:** 16;
- **initial_lr:** 0.0002;
- **criterion:** nn.L1Loss();
- **optimizer:** optim.Adam(model.parameters(), lr=lr, weight_decay=0, betas=(0.9, 0.99));
- **scheduler:** CosineAnnealingLR(optimizer, T_max=epochs, eta_min=1e-7).

3.2 Training

The averaged L1 loss in each epoch was recorded and plotted in Figure 1.

3.3 Validation

After each epoch, the model at that time evaluated the 400 images in the validation dataset, and the (average) PSNR score was recorded and plotted in Figure 2. **The model (i.e., weights) that yields the highest validation PSNR is returned as the final trained model.**

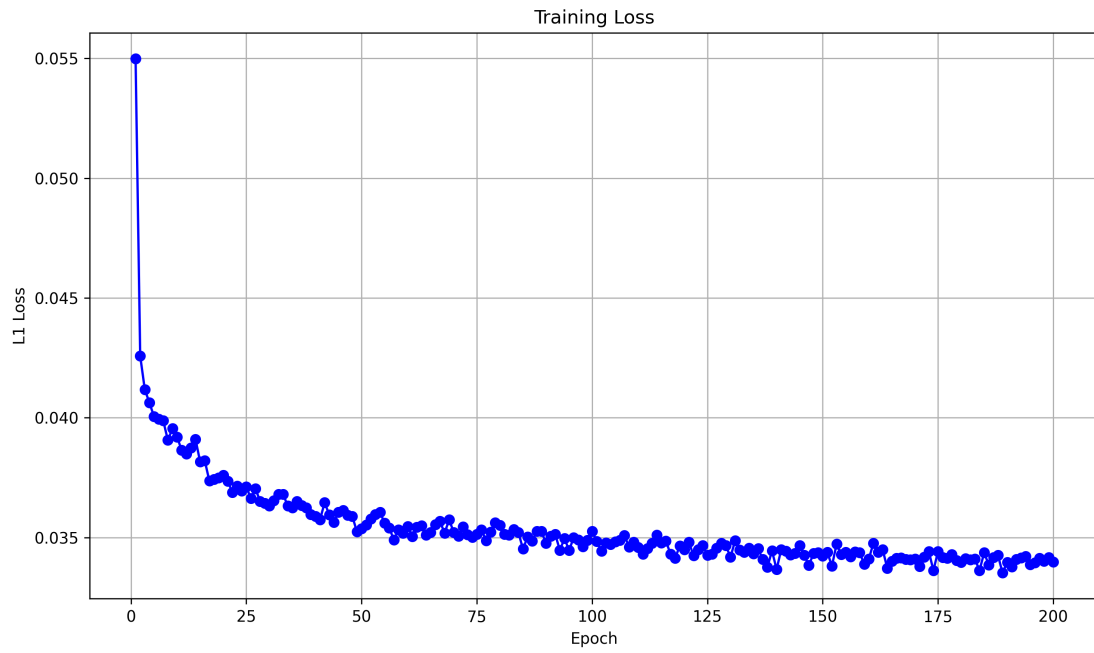


Figure 1: Training loss in 200 epochs.

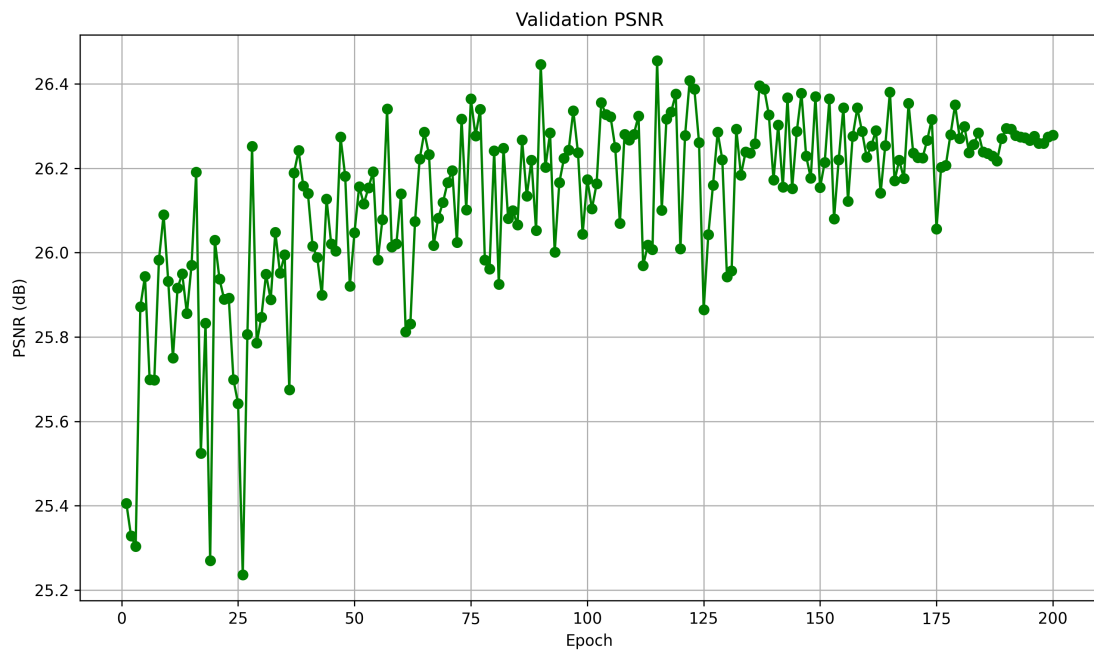


Figure 2: Validation PSNR in 200 epochs.

3.4 Specs

The model is trained using A100 GPU on Google Colab.

4 Codalab Evaluation

The model was then used to infer the 400 test images on Codalab, and the screenshot of the PSNR score obtained is shown in Figure 3.

Here are your submissions to date (✓ indicates submission on leaderboard):

#	SCORE	FILENAME	SUBMISSION DATE	SIZE (BYTES)	STATUS	✓	
1	26.6124114982	submission_1.zip	04/25/2025 04:18:48	71302	Finished		+
2	26.6143684107	preds.zip	04/25/2025 12:14:29	77178	Finished	✓	+

Figure 3: Codalab score

A model.py

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class CALayer(nn.Module):
    """Channel Attention (squeeze & excite)"""
    def __init__(self, channel, reduction=16):
        super().__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Sequential(
            nn.Conv2d(channel, channel // reduction, 1, bias=True),
            nn.ReLU(inplace=True),
            nn.Conv2d(channel // reduction, channel, 1, bias=True),
            nn.Sigmoid()
        )
    def forward(self, x):
        w = self.avg_pool(x)  # [B, C, 1, 1]
        w = self.fc(w)        # [B, C, 1, 1]
        return x * w

class RCAB(nn.Module):
    """Residual Channel-Attention Block"""
    def __init__(self, nf, reduction=16):
        super().__init__()
        self.body = nn.Sequential(
            nn.Conv2d(nf, nf, 3, 1, 1, bias=True),
            nn.ReLU(inplace=True),
            nn.Conv2d(nf, nf, 3, 1, 1, bias=True),
            CALayer(nf, reduction)
        )
    def forward(self, x):
        return x + self.body(x)

class RCAN(nn.Module):
    def __init__(self, in_nc=3, out_nc=3, nf=64, nb=20, scale=4):
        super().__init__()
        # head
        self.head = nn.Conv2d(in_nc, nf, 3, 1, 1, bias=True)
        # body: nb RCABs + one conv
        body = [RCAB(nf) for _ in range(nb)]
        body.append(nn.Conv2d(nf, nf, 3, 1, 1, bias=True))
```

```

self.body = nn.Sequential(*body)
# tail (upsampling)
tail = []
for _ in range(int(scale // 2)):
    tail += [
        nn.Conv2d(nf, nf * 4, 3, 1, 1, bias=True),
        nn.PixelShuffle(2),
        nn.ReLU(inplace=True)
    ]
tail.append(nn.Conv2d(nf, out_nc, 3, 1, 1, bias=True))
self.tail = nn.Sequential(*tail)

def forward(self, x):
    x = self.head(x)
    res = self.body(x)
    x = x + res
    x = self.tail(x)
    return x

```